



CMC UNIVERSITY

Aspire to Inspire the Digital World

Machine Learning and Data Mining

Faculty of Information and Communication Technology

2.1. Convolutional Neural Network

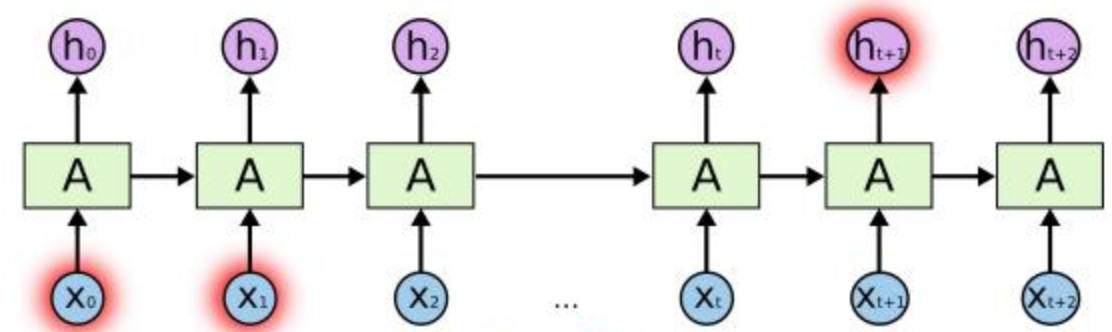
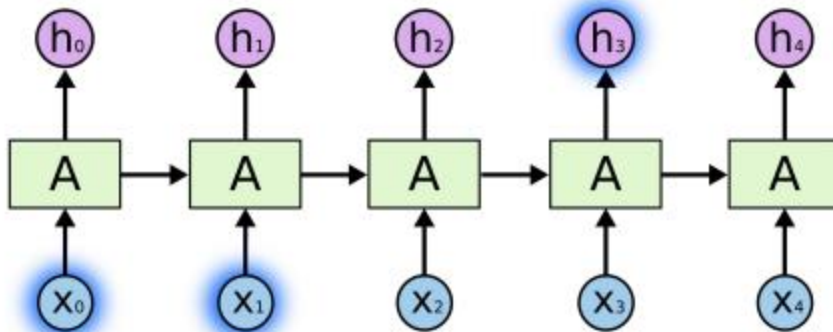
2.2 Recurrent Neural Network

2.3 Long-Short Term Memory

2.4 Optimization techniques

Long-Short Term Memory

- A small example where RNN can work perfectly :
 - Prediction of the last word in the sentence : “The clouds are in the sky”
- RNN can't handle situation where the gap between the relevant information and the point where it is needed is very large.



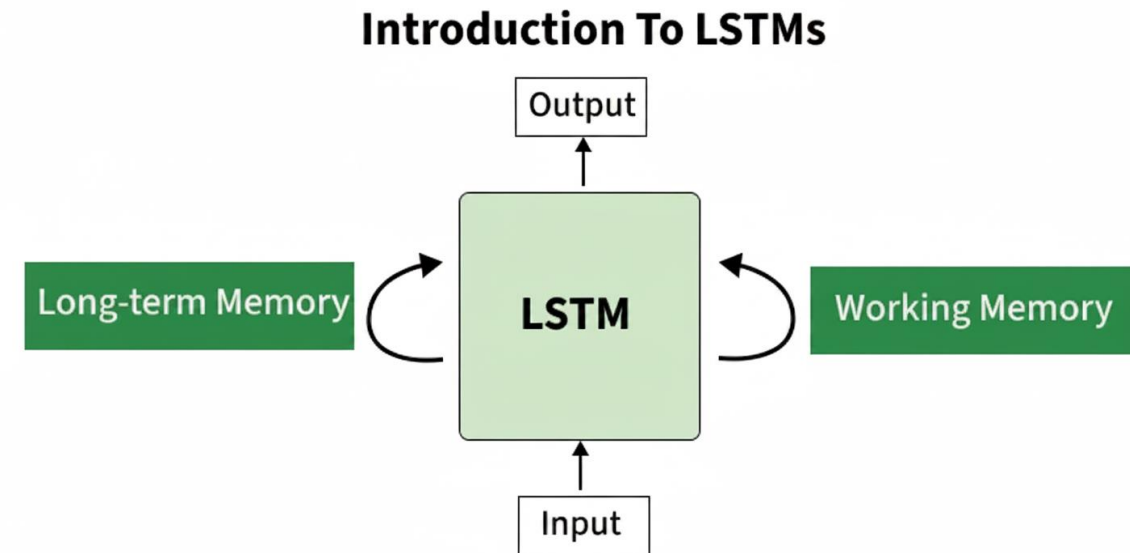
Long Short-Term Memory (LSTM) is an improved version of Recurrent Neural Network (RNN) designed to capture long-term dependencies in sequential data. It uses a memory cell to store information over time, solving the limitations of traditional RNNs.

Handles Long Term Dependencies: Remembers information for longer sequences

Memory Cell: Stores and updates important information over time

Better than RNN: Overcomes short term memory limitations

Applications: Used in language translation, speech recognition and time series forecasting



Picture from Geekforgeeks.com

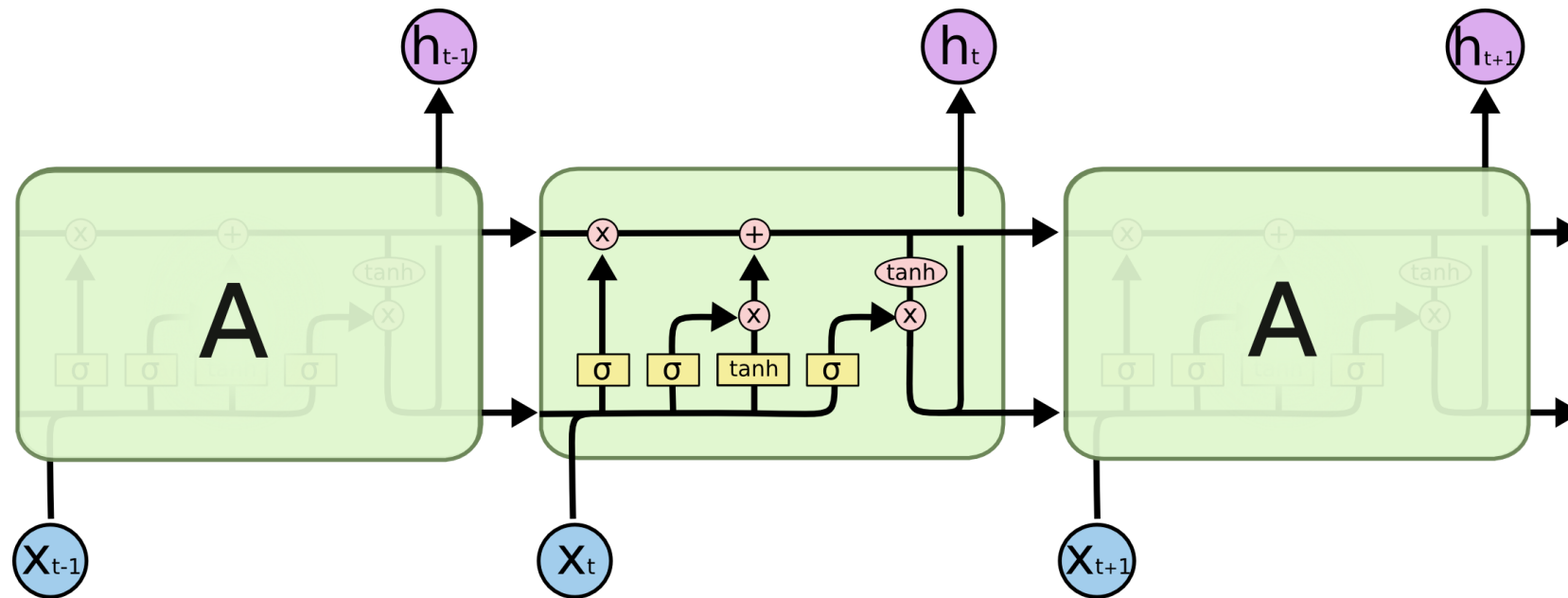
LSTM Architecture

LSTM architectures involves the memory cell which is controlled by three gates:

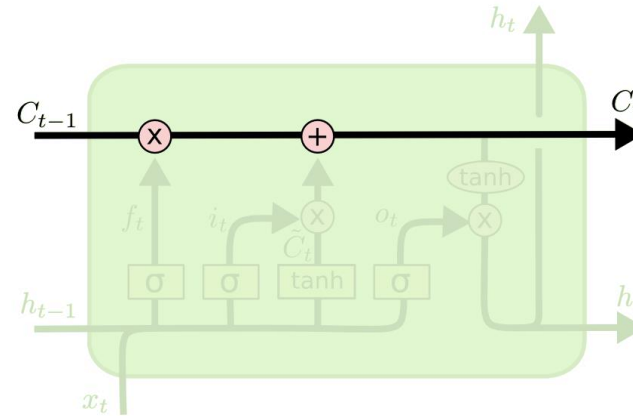
Input gate: Controls what information is added to the memory cell.

Forget gate: Determines what information is removed from the memory cell.

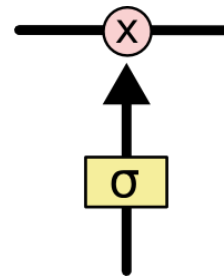
Output gate: Controls what information is output from the memory cell.



The core idea behind LSTMs is the cell state.



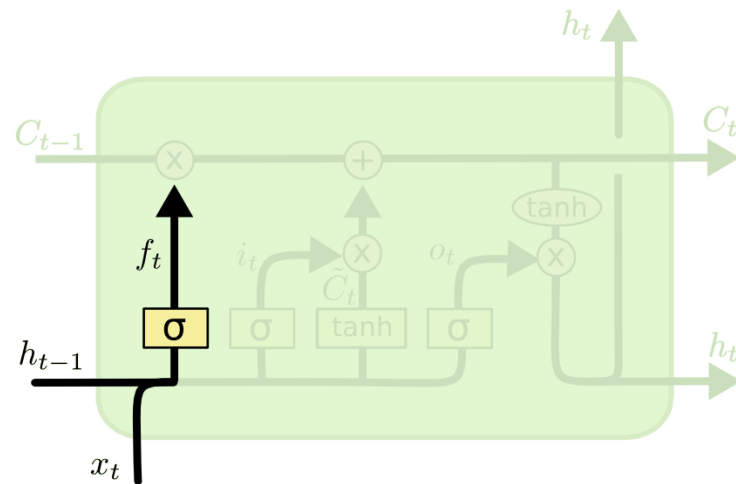
The LSTM has the ability to remove or add information to the cell state: thanks to gates



Gates are generally composed out of a sigmoid neural net layer and a pointwise multiplication operation

Step-by-Step LSTM Walk Through

Step 1: Decide what information to throw away from the cell state, forget layer.

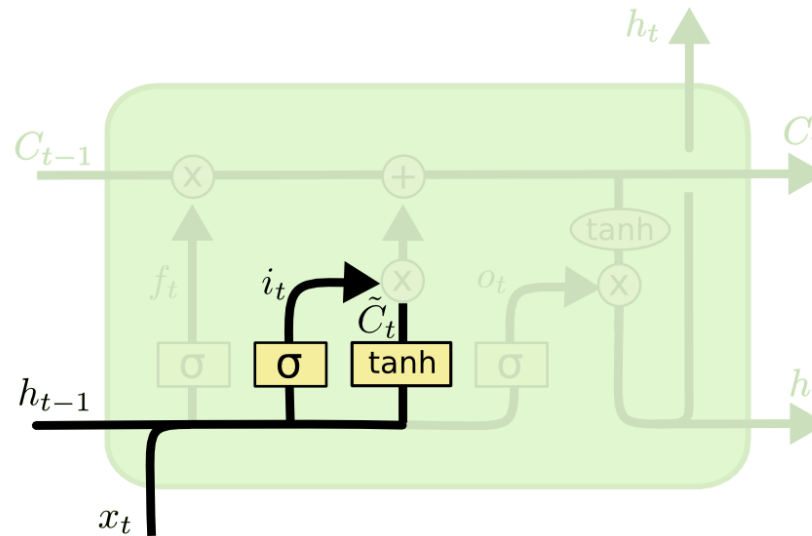


$$f_t = \sigma (W_f \cdot [h_{t-1}, x_t] + b_f)$$

- 1 represents “completely keep this”
- 0 represents “completely get rid of this.”

Step-by-Step LSTM Walk Through

Step 2: Decide what new information we're going to store in the cell state.



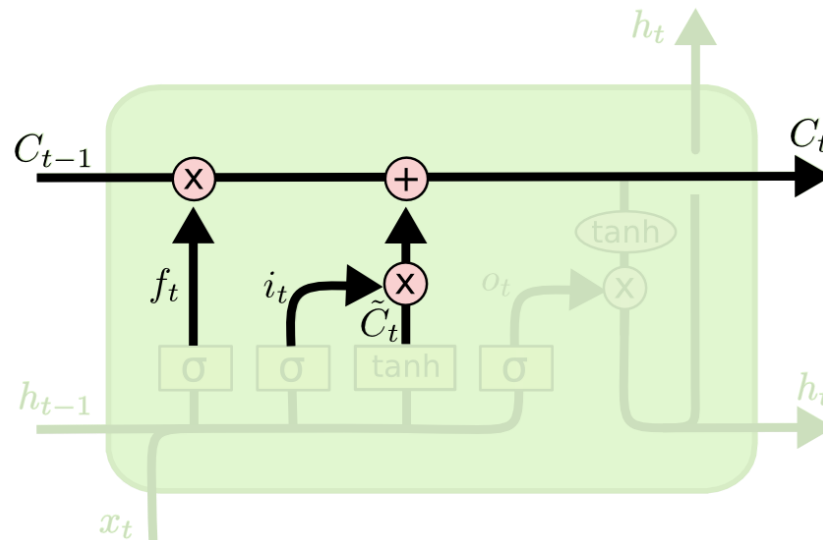
$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

- Input gate layer : decides which values we will update
- Tanh layer : creates a vector of new candidate values

Step-by-Step LSTM Walk Through

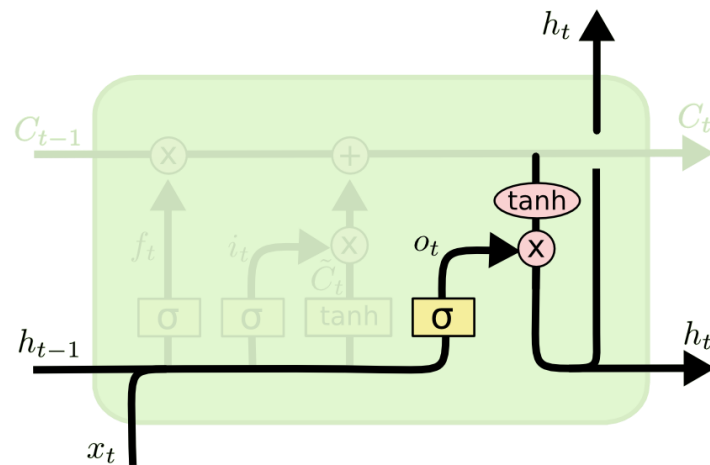
Step 3: Update the cell state



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Step-by-Step LSTM Walk Through

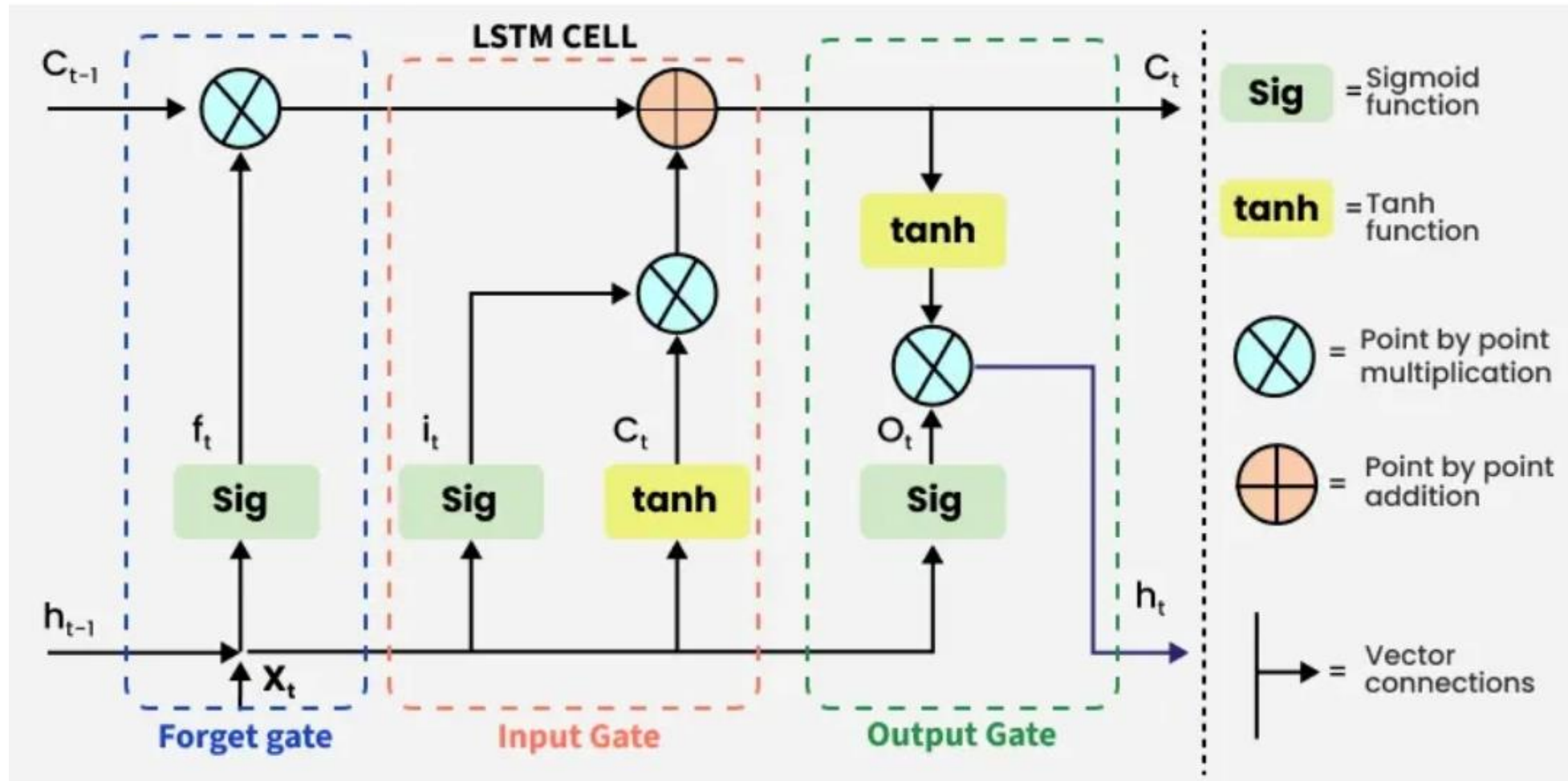
Step 4: Decide what is the output



$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

LSTM structure



LSTM sample: Application of RNN/LSTM in education.

Knowledge tracing: modeling student knowledge over time so that we can accurately predict how students will perform on future interactions.

Time	Question (Skill)	Answer (result)
t ₁	Addition	False
t ₂	Addition	True
t ₃	Subtraction	False
t ₄	Addition	True
t ₅	? (Addition)	?

LSTM sample: Application of RNN/LSTM in education.

Knowledge tracing: modeling student knowledge over time so that we can accurately predict how students will perform on future interactions.

$$x_t = (\text{skill}, \text{result})$$

Encoding:

Addition + correct $\rightarrow (1,1)$

Addition + wrong $\rightarrow (1,0)$

Subtraction + wrong $\rightarrow (2,0)$

Input sequence:

$$x_1, x_2, x_3, x_4$$

LSTM sample: Application of RNN/LSTM in education.

LSTM Learning:

- Are students understanding Addition?
- Does knowledge improve over time?
- Is it forgotten?

Via:

$$h_t, C_t$$

Predict at the next step: $h_4 \rightarrow \hat{y}_5$

In which: $\hat{y}_5 = P(\text{correct at } t_5)$

LSTM sample: Application of RNN/LSTM in education.

```
# Simple Knowledge Tracing with LSTM (PyTorch)
import torch
import torch.nn as nn

# Example data: (skill_id, correct)
# skill: 0=Addition, 1=Subtraction
sequence = [(0,0),(0,1),(1,0),(0,1)]

# Convert to tensor (one-hot simple encoding)
def encode(seq):
    data = []
    for skill, result in seq:
        vec = [0]*4
        vec[skill*2 + result] = 1
        data.append(vec)
    return torch.tensor(data,
dtype=torch.float32)
```

LSTM sample: Application of RNN/LSTM in education.

```
x = encode(sequence).unsqueeze(0) # batch=1

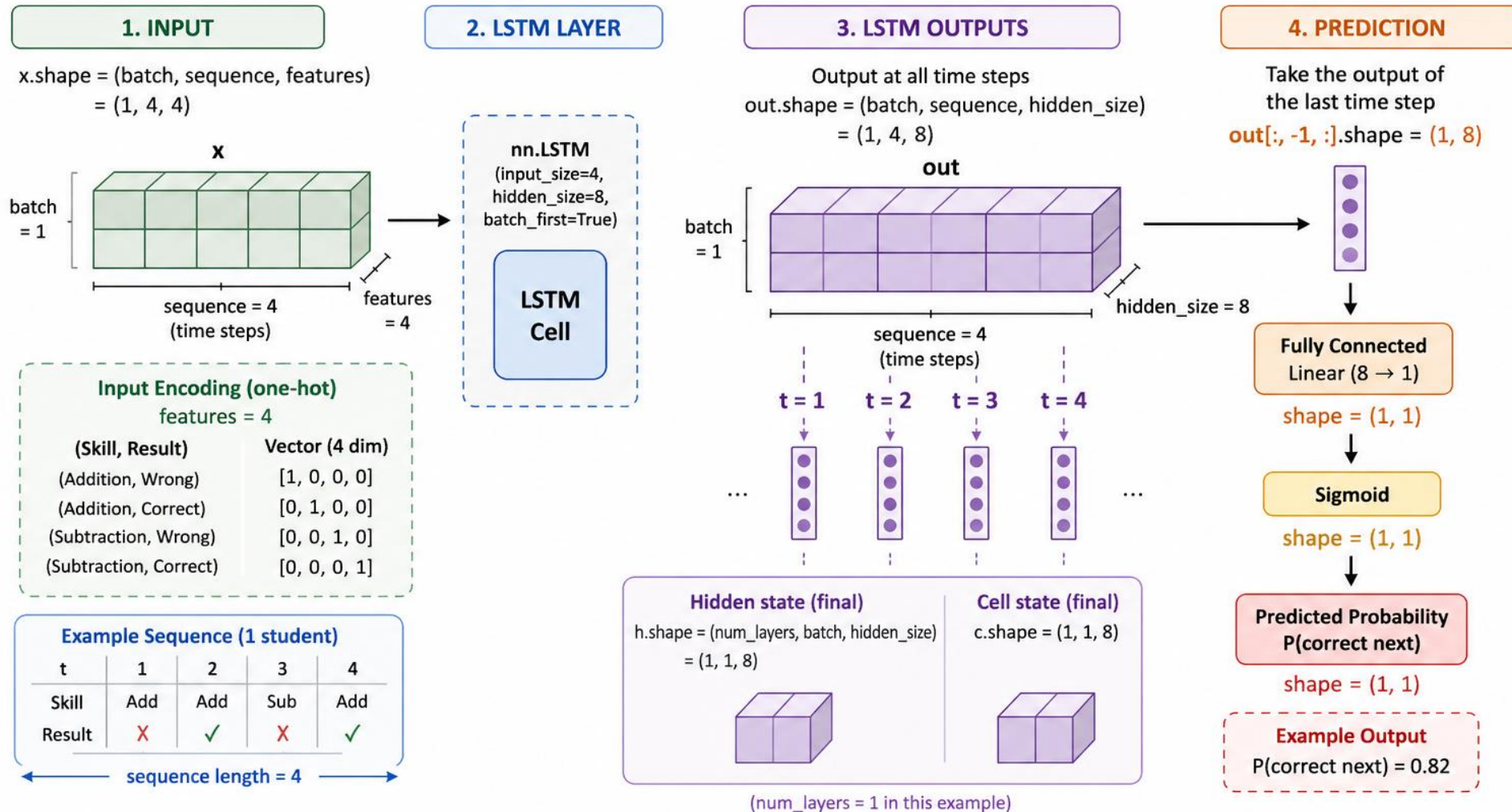
class KTModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.lstm = nn.LSTM(input_size=4, hidden_size=8, batch_first=True)
        self.fc = nn.Linear(8, 1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        out, _ = self.lstm(x)
        out = self.fc(out[:, -1, :])
        return self.sigmoid(out)
```

LSTM sample: Application of RNN/LSTM in education.

```
model = KTModel()  
  
# Forward pass  
y_pred = model(x)  
  
print("Predicted probability of correct next  
answer:", y_pred.item())
```

Predicted probability of correct next answer: 0.5177015066146851

LSTM Tensor Flow for Knowledge Tracing ($\text{batch_first}=\text{True}$)

LEGEND

- **batch (B):** number of students in a batch
- **sequence (T):** number of interactions (time steps)
- **features (F):** input feature dimension (here = 4)
- **hidden_size (H):** LSTM hidden dimension (here = 8)
- **num_layers (L):** number of stacked LSTM layers (here = 1)

Shape Notation

 $(B, T, F) \rightarrow (B, T, H) \rightarrow (B, 1) \rightarrow (B, 1)$